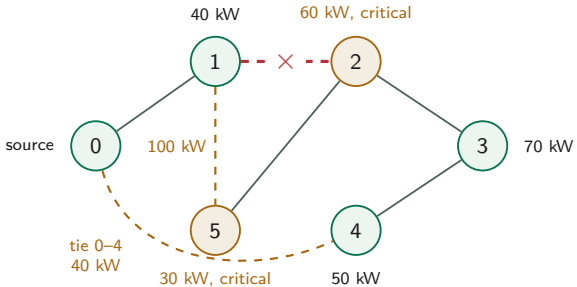
A fault changes the paths that are available



A small, transparent fixture

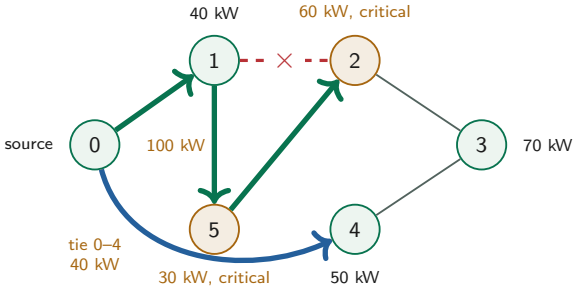
Six nodes; five loads; one failed segment; two optional ties. Amber nodes have higher service priority.

Predict: Can we simply reconnect the failed segment?

Explanation: No. Its availability is fixed to zero. Planning must find another admissible path.

Fresh classroom fixture: Classroom_Labs L6; src/standard_formulations.py. Restoration context: Chen et al., IEEE TPWRS, 2018.

Two ties create choices, with different bottlenecks



Follow the power

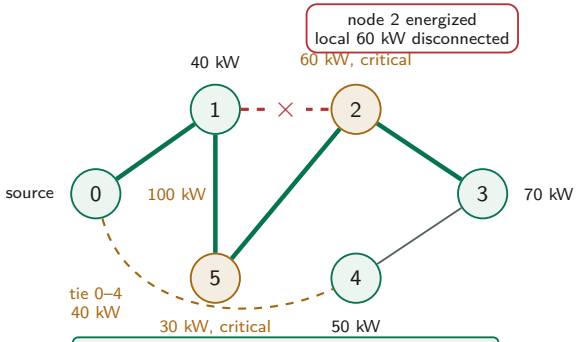
The route $0 \to 1 \to 5 \to 2$ uses the 100 kW tie. The other tie has only 40 kW in this first case.

Predict: Can the 40 kW tie pick up the entire 50 kW load at node 4? No: $50 > 40$. Loads are indivisible here; partial pickup is excluded.

Fixture and binary pickup assumption: Classroom_Labs L6; standard_formulations.restoration_graph. Blue and teal paths are explanatory alternatives, not a switching sequence.

Next: Does an energized node have to serve its local load?

A node can carry power while its load stays disconnected



Two different binary decisions

$y_i = 1$: node i is energized.
$s_i = 1$: its load is picked up.

We require $s_i \leq y_i$.

Predict: Why is equality $s_i = y_i$ too restrictive?

Explanation: It would forbid a useful transit node whose own load remains shed.

Original schematic of a feasible six-node plan; binary pickup/energization model in src/standard_formulations.py.

A capacity budget turns priorities into a real choice

Critical pair: $60 + 30 = 90$ kW





Largest total: $70 + 30 = 100$ kW

The 100 kW tie is the budget

Both choices also serve node 1's 40 kW upstream.
Critical pair: total 130; critical 90.
Largest total: total 140; critical 30.

Predict: Which outcome should a critical-service policy prefer?

Explanation: The answer depends on the stated objective, not on how persuasive its explanation sounds.

Hand-checkable choices at tie 0–4 capacity 40 kW; exact optima confirmed by Classroom_Labs L6. Bars represent kW, not physical line lengths.

Compare outcomes before comparing algorithms

Same 40 kW tie case	Total served	Critical served	Critical share served
Maximize total kW	140 kW	30 kW	30/90
Maximize weighted kW	130 kW	90 kW	90/90

What the experiment establishes

Changing the service objective changes who benefits. It does not establish an advantage caused by an LLM or by the word "agentic."

Predict: What must stay matched to isolate an agentic workflow effect?

Explanation: The objective, candidate tools, physical limits, information and verification rules.

Fresh six-node experiment: Classroom_Labs L6, weighted MILP versus unweighted exact enumeration; both use the same feasible constraints.

The objective is an explicit, inspectable service policy

$$\max \sum_{i=1}^{5} w_i d_i s_i, \qquad w_2 = w_5 = 10, \qquad w_1 = w_3 = w_4 = 1.$$

Critical-pair choice

$10(60 + 30) + 40 = \mathbf{940}$
weighted kW; 130 actual kW.

Maximum-total choice

$10(30) + 70 + 40 = \mathbf{410}$
weighted kW; 140 actual kW.

d_i: load demand in kW; w_i: dimensionless priority; $s_i \in \{0, 1\}$: pickup.

Predict: Does weight ten guarantee critical-first service on every possible network?

Explanation: No. Strict priority requires a lexicographic objective or a proven sufficiently large weight.

Classroom priority policy: standard_formulations.restoration_graph. Weighted restoration objectives: Chen et al., 2018.

Planning: binary pickup and topology decisions

Symbol	Meaning
$s_i \in \{0,1\}$	load i picked up
$y_i \in \{0,1\}$	node i energized
$z_e \in \{0,1\}$	edge e closed
$P_e \in \mathbb{R}$	signed active flow, kW
$\bar{P}_e$	edge capacity, kW

$$s_i \le y_i, \qquad y_0 = 1,$$
$$z_{ij} \le y_i, \qquad z_{ij} \le y_j,$$
$$z_{1,2} = 0.$$

Interpret each line

Picked-up loads need energized nodes; closed edges need energized endpoints; the fault stays isolated.

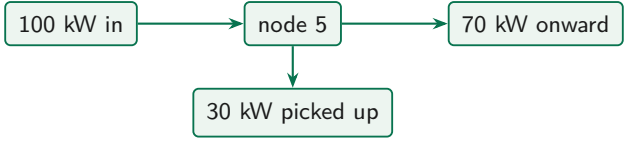
Predict: Which of these rules prevents a disconnected energized island?

Explanation: None yet. Connectivity must be imposed separately.

Fresh MILP implementation: src/standard_formulations.py. Radiality and connectivity: Wang et al., 2020.

Power balance says where every picked-up kilowatt comes from

$$\underbrace{\sum_{e} A_{ie} P_e}_{\text{net active power entering node } i} = \underbrace{d_i s_i}_{\text{picked-up demand}}, \quad i \neq 0, \qquad -\bar{P}_e z_e \leq P_e \leq \bar{P}_e z_e.$$



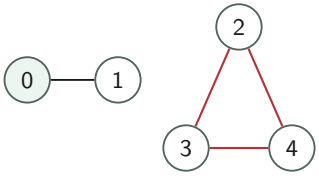
$A_{ie} = +1$ for an edge entering i, -1 leaving it, zero otherwise.

Know the model boundary

This is lossless active-power transport. Voltage, reactive power, AC losses and protection are not certified.

Classroom_Labs L6; active-transport formulation in src/standard_formulations.py. This six-node model is separate from the archived feeder power-flow checks.

Edge counts alone do not prove a source-connected tree



5 nodes; 4 edges
but an unsupplied cycle

$$\sum_e A_{ie} f_e = y_i \qquad (i \neq 0),$$

$$-5z_e \leq f_e \leq 5z_e,$$

$$\sum_e z_e = \sum_i y_i - 1.$$

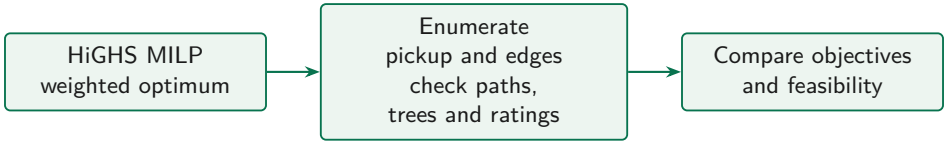
f_e: fictitious connectivity flow, not electrical kW. Every energized non-source node consumes one unit. Five is the maximum number of such nodes.

Two jobs, two conditions

Commodity flow proves reachability. Reachability plus the edge count gives a tree on the energized nodes.

Original counterexample; single-commodity connectivity and radiality: Wang et al., 2020; classroom MILP.

Verification uses an independent, exhaustive small-case check



Two solves within the MILP method

First maximize weighted service. Then preserve that optimum and minimize selected edge count, with a small edge-index tie-breaker.

Predict: Does minimizing selected edges minimize switching operations?

Explanation: No. Operation count needs the initial switch states and a sequence-aware model. Exhaustive enumeration is practical because this fixture has only six nodes. It is a verification aid, not a proposed large-feeder algorithm.

Classroom_Labs L6; standard_formulations.solve_restoration_milp and enumerate_restoration.

Hands-on: vary the bottleneck, keep the policy visible

Notebook L6 Eight minutes: run the full cell, then compare 40, 80 and 100 kW.

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
timings['L6 fresh MILP sweep + independent enumeration'] = time.perf_counter() - started
display(restoration['table'].drop(columns='pickup'))
```

Before running

Predict total and critical pickup at 40 kW. Then predict whether every increase in capacity must increase service.

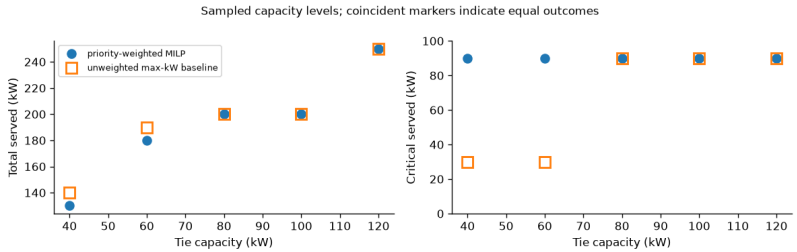
Read the result

Confirm MILP/enumeration agreement. Record both service measures and the elapsed solve/check time. Explain one plateau.

Run the entire L6 cell; the displayed excerpt only locates the experiment. Samples are discrete capacities, not a continuous response curve.

Exact excerpt: Classroom_Labs.ipynb, L6 lines 1–5. Fresh results use the six-node active-transport fixture.

More capacity can produce the same discrete pickup



Two ratings, same pickup

At 80 and 100 kW: weighted service is 200 kW total, including all 90 critical kW.

Predict: Is the flat step evidence that capacity has no value?

Explanation: No. It reflects indivisible load blocks and the available routes at these sampled ratings.

Original computed figure: Classroom_Labs L6 / classroom_restoration.png. Same feasible constraints; weighted versus unweighted objectives.

The workflow surrounds the solver with evidence and decisions

Stage	Specific method	Output that can be checked
Awareness	Topology and status checks	Fault 1–2; ratings; critical nodes 2, 5
Planning	Weighted restoration MILP	Pickup vector and selected edges
Simulation	Active-power transport	Per-edge demand and capacity margins
Verification	Independent enumeration	Feasible and objective agreement
LLM Optimizer	Allowed refinement request	Proposed next calculation, with rationale

Scope matters

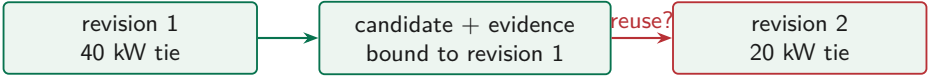
L6–L7 run fresh numerical and binding checks. L10 replays the full five-stage language trace from the separate archived feeder experiment.

Classroom_Labs L6–L7, L10; proposal_complete_workflow.json. This table describes stage roles; L6 does not make five live LLM calls.

Hands-on: old evidence does not certify a changed rating

Notebook L7 Four minutes: run the full cell, then explain the two Boolean results.

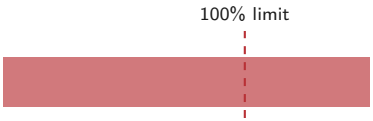
```
new_rest_request = {**rest_request, 'capacity_kw': 20., 'revision': 2}
print('Original contract:', lab.bound_to(rest_request, rest_candidate, rest_evidence))
print('Changed rating with old evidence:', lab.bound_to(new_rest_request, rest_candidate, rest_evidence))
print('Required response: HOLD old recommendation; request a fresh constrained solve.')
```



Explanation: Original binding: True. Changed request with old evidence: False. HOLD and re-solve.

Exact excerpt: Classroom_Labs L7 lines 7–10; lab.bind_evidence and lab.bound_to. Binding checks identity, not physical feasibility by itself.

Archived feeder: an attractive pickup can fail a thermal check



Verification rejects the initial high-pickup proposal.

Different fixture; explicit scope

Archived MV feeder; synthetic short bypass; 100 A limit; ideal LV aggregation. No time-ordered switching certificate.

Refine, simulate, verify again

A permitted shedding refinement produces a passing candidate: 1843.61 kW served, including 474.04 kW critical.

ARCHIVED ACTUAL: proposal_complete_workflow.json, restoration / lindistflow3. Initial E01 loading 151.639%; final verified SHED_075.

Llama can request a useful check and still rank results incorrectly

Actual Planning response: 0.874 s

"Requesting both initial simulations as per the Planning Agent's role and the contract's operational limit."

Actual LLM ranking: numerical guard required

Returned ranking: E02, E03.
Returned scores: E02 $= -5509.14$; E03 $= -6109.97$.
The task minimizes cost, so **E03 must precede E02**.

Predict: Should a majority of similar language responses overrule the score?
No. Recompute the ordering from verified numerical evidence.
The accepted proposal is tied to a fresh final verification.

Exact Planning quote and returned ranking: archived restoration / lindistflow3 trace, proposal_complete_workflow.json. Times are recorded model-call wall times.

Matched supervision: equal service, different latency

Archived LinDistFlow3 restoration	Fixed supervisor	Llama supervisor
Total served (kW)	1843.61	1843.61
Critical served (kW)	474.04	474.04
Objective cost	−6109.97	−6109.97
Total elapsed time (s)	0.306	5.232

A result worth teaching honestly

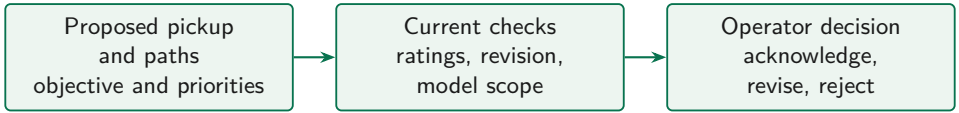
This run shows no service improvement from Llama. Its visible contribution is a tool-selection rationale and trace; it adds latency and needs numerical guards.

Predict: What would demonstrate an additional agentic benefit?

Explanation: Matched tests of changed requests, tool failures and stale evidence, with success rate and human effort measured.

ARCHIVED ACTUAL: fixed versus agentic, restoration / lindistflow3, proposal_complete_workflow.json. Single recorded case; four Llama calls.

Handoff: show the action, the evidence and the remaining decision



A complete recommendation is conditional

State which measurements and equipment statuses were trusted. A passing calculation is tied to that information and that model.

Predict: What if the network model is correct but the measurements are deceptive?

Explanation: The planner may optimize the wrong state. The next case tests consistency, observability and trusted replacement evidence.

Restoration lessons: Classroom_Labs L6–L7 and archived L10 trace. Operator acknowledgment is a tutorial workflow, not a field execution system.